

1.3 Weitere Listenmethoden - Lösung

Neben add(DataElement data), size(), print() und remove(DataElement compare) verfügen Listen noch über weitere Methoden. Besprechen Sie mit ihrem Nachbarn wie die Methoden umgesetzt werden können. Notieren Sie Pseudocode und markieren / notieren Sie Schwierigkeiten bei der Umsetzung. Notieren Sie während der Implementierung auch Ihren Java Code.

0. public boolean removeHead() { //in der Klasse List

```
// TODO 0 come on... you know how to do this!
if (head != null) {
    head = head.getSuccessor();
    return true;
} else {
    return false;
}}
```

1. public boolean removeTail() { //in der Klasse Node

```
// TODO 1-1 check if successor's successor is not null
// recursive call
// remove reference to successor; true
if(successor.getSuccessor() != null) {
    return successor.removeTail();
} else {
    successor = null;
    return true;
}}
```

2. public boolean addHead(DataElement data) { //in der Klasse List

```
// TODO 1-0 check if head is not null
// call on head
// false
if (head != null) {
    return head.removeTail();
} else {
    return false;
}}
```

3. public boolean addBefore(DataElement data, DataElement compare) { //in der Klasse Node

```
// TODO 3-1 check if successor is not null
// check if successor's DataElement is the one we want to compare
against
// create temp Node from data; set references to successors on
// successor and this Node; true
// recursive call
// false
if(successor != null ) {
    if(successor.getDataElement().equals(compare)) {
        Node temp = new Node(data);
        temp.setSuccessor(successor);
        successor = temp;
        return true;
    } else {
        return successor.addBefore(data, compare);
    }
} else {
    return false;
}}
```

4. **sortedInsert(DataElement data)** = hier müssen zum Teil umfangreichere Methoden in mehreren Klassen angepasst werden. Notiere zunächst die Klassen.

smallerThan(DataElement data) in Interface DataElement

```
boolean smallerThan(DataElement data); //TODO 4-0 in subclasses
```

In der Klasse List:

```
public boolean sortedInsert(DataElement data) {
    // TODO 4-0 check if head is not null
    //     check if head's dataElement is smaller than data
    //     recursive call
    //     well.. what would be the easiest way to add a head to this List?
    //     false
    if (head != null) {
        if (head.getDataElement().smallerThan(data)) {
            return head.sortedInsert(data);
        } else {
            return addHead(data);
        }
    } else {
        return false;
    }
}
```

In Node

```
public boolean sortedInsert(DataElement data) {
    // TODO 4-2 check if successor's dataElement is smaller than the given data
    //     recursive call
    //     temporary Node from data; cleanup successor references; true
    if (successor.getDataElement().smallerThan(data)) {
        return successor.sortedInsert(data);
    } else {
        Node temp = new Node(data);
        temp.setSuccessor(successor);
        successor = temp;
        return true;
    }
}
```

In Customer oder Patient

```
@Override
public boolean smallerThan(DataElement data) {
    if (data.getClass().equals(this.getClass())) {
        Customer customer = (Customer) data;
        // TODO 4-1 use <StringObject>.compareTo(<anotherStringObject>) method
        // to compare the two Strings.
        //     if the returned value is < 0 the <StringObject> is
        //     lexicographically smaller; true
        int compareValue = name.compareTo(customer.getName()) +
        birthday.compareTo(customer.getBirthday());
        return compareValue < 0;
    } else {
        System.out.println("ERROR: incompatible types!");
        return false;
    }
}
```